LLM fine-tuning study guide

Pretraining, supervised fine-tuning (SFT), PEFT, instruction tuning, and alignment with human feedback

> **How to read this.** Your class notes are the backbone of this guide. Items marked **(added)** are extra explanations or facts that were not in your screenshots. Items marked **(check)** are places where the notes and public sources differ, or where the notes were unclear.

1. The big picture

An LLM is a transformer trained in stages. Each stage changes what the model is good at.

1 Pretraining
Unsupervised / self-supervised

>

2 SFT
Supervised fine-tuning

>

3 Alignment
Human feedback: RLHF, DPO, RLAIF

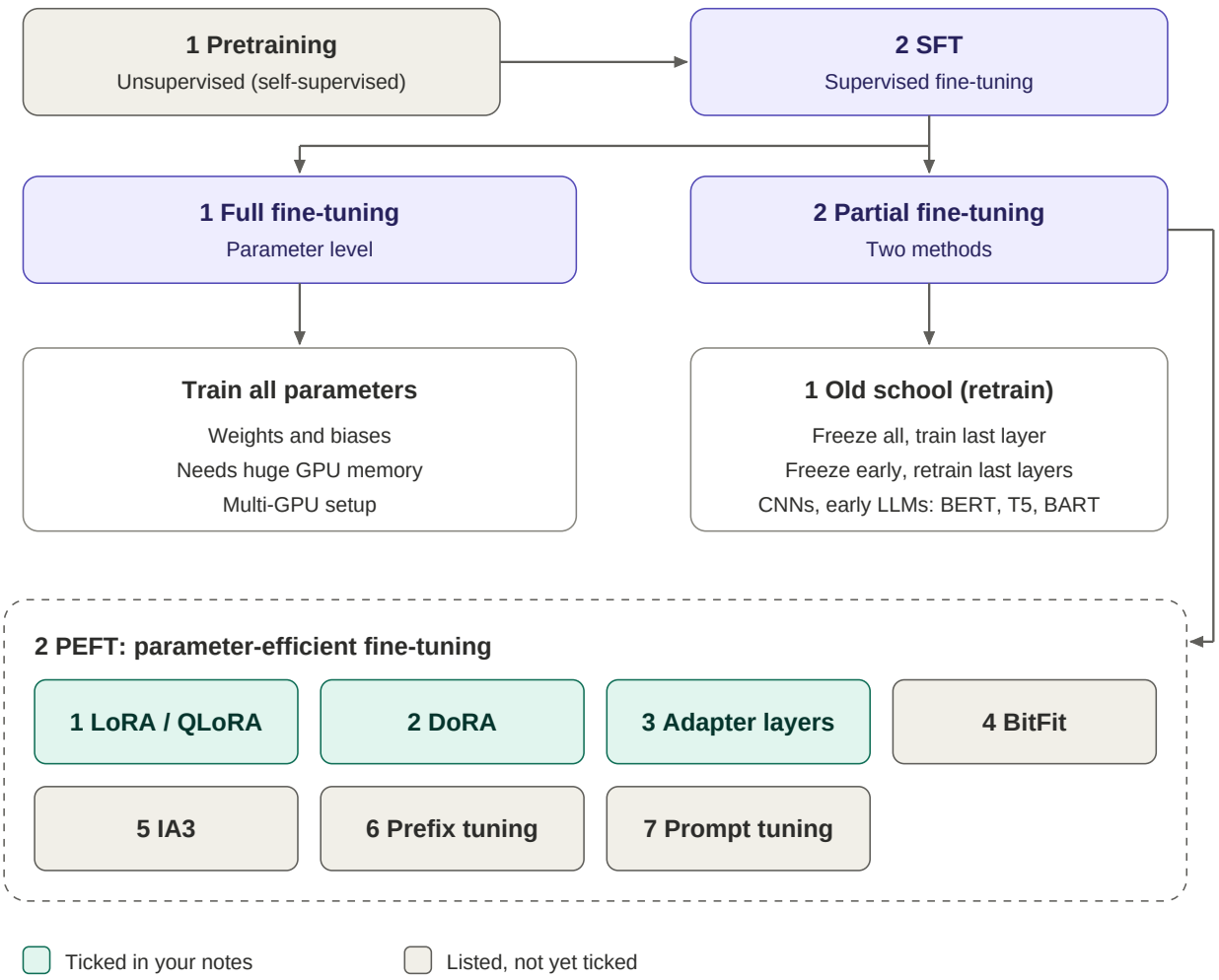
Stage	Data	Objective	Result
1 Pretraining	Huge raw text: web, books, papers, code	Predict the next token	Base model: knows language and facts, but does not follow instructions
2 SFT	Curated labeled data (plain text or instruction + response)	Supervised loss on the target text	Domain expert and/or chatbot-style assistant
3 Alignment	Ranked pairs of responses (better vs worse)	Prefer the better response	Polite, safe, helpful model aligned with human values

SFT can be described along two independent axes. Keep them separate in your head:

- **Parameter level**: *which weights do we update?* (full fine-tuning vs partial fine-tuning, which includes PEFT).
- **Data level**: *what does the training data look like?* (non-instructional, instructional, or preference data for alignment).

> **Key idea (added).** LoRA, adapters and the like tell you *how* to train cheaply. Non-instructional vs instructional tells you *what* to train on. You can combine them, for example instruction tuning a model with QLoRA.

Diagram 1: fine-tuning map



Teal = covered in class (ticked); gray = listed but not yet covered.

2. Stage 1: pretraining

- **Type of learning:** unsupervised, also called self-supervised learning. The labels come from the text itself (the next token), so no human labeling is needed.
- **Objective:** next-token prediction (language modeling).
- **Sources of data:** documentation; research papers (ArXiv, PubMed); Common Crawl (large-scale web data); Wikipedia (English and other languages); books (Book Corpus).
- **Result:** a base model, for example LLaMA-2 (7B, 13B, 70B), Mistral, GPT, DeepSeek, Gemini.
- **After this stage:** the model has knowledge and grammar, but it still lacks the ability to follow instructions, keep a polite tone, or give structured answers.

Why is it called self-supervised? (added)

Given the text "Sunny Savita where AI meets", the training target for the next position is simply the next word in the document. The data supplies its own answer, so there is no separate label column.

3. Stage 2: supervised fine-tuning at the parameter level

Full fine-tuning

- Train **all** parameters (weights and biases).
- Needs huge GPU memory and a multi-GPU setup.
- **(added)** Why so heavy: with mixed-precision training and the Adam optimizer you store weights, gradients and optimizer states. A common estimate is about 16 bytes per parameter, so a 7B model needs on the order of 100

GB before activations.

- **(added)** Risk: catastrophic forgetting, where the model loses general ability while adapting to new data.

Partial fine-tuning: two methods

Method 1: old school (retrain). Used with CNN-based models and early-stage LLMs such as BERT, T5 and BART.

- Freeze all layers and train only the last output layer.
- Or freeze some starting layers and retrain some of the last layers.

Method 2: PEFT (parameter-efficient fine-tuning). Keep the base model frozen (or nearly) and train a very small number of extra or selected parameters. The seven techniques in your notes:

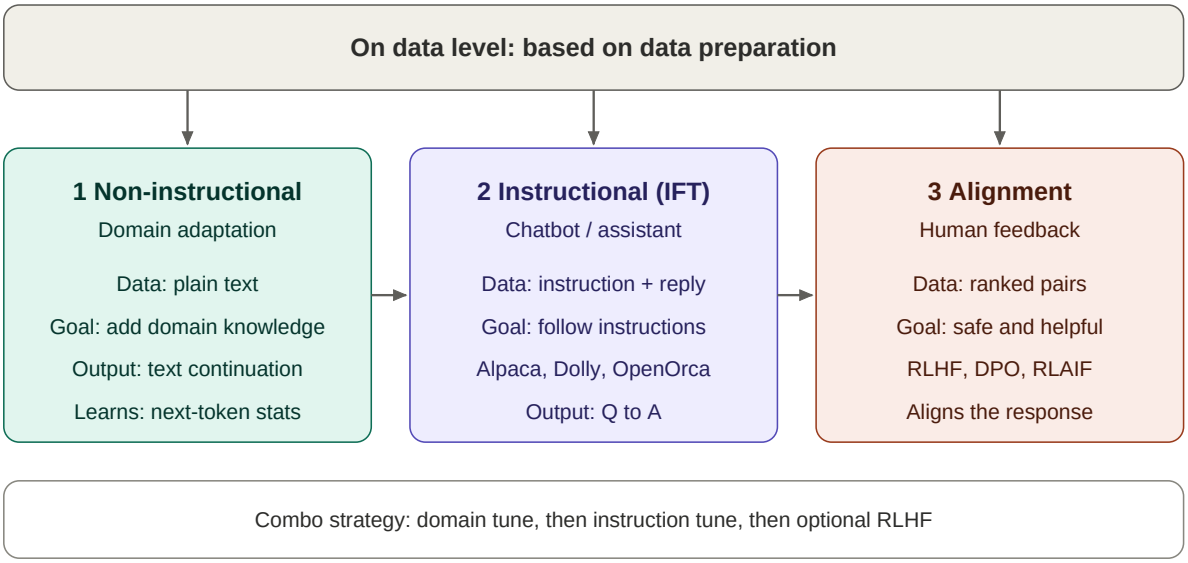
#	Technique	What it trains (added)	Notes
1	LoRA / QLoRA	Small low-rank matrices A and B added beside frozen weight matrices, so the update is W + BA. QLoRA does the same on a base model loaded in 4-bit precision.	Your notes: lower-precision model and memory-efficient loading (this is the QLoRA idea). Covered in class.
2	DoRA	Splits each weight into magnitude and direction; applies LoRA to the direction and trains the magnitude separately.	Covered in class. Often a bit more accurate than LoRA.
3	Adapter layers	Small bottleneck modules (down-project, non-linearity, up-project) inserted inside each transformer block.	Covered in class. Adds a little inference latency.
4	BitFit	Trains only the bias terms of the model.	Extremely few parameters.
5	IA3	Learns scaling vectors that rescale keys, values and feed-forward activations.	Even fewer parameters than LoRA.
6	Prefix tuning	Learns trainable vectors prepended to the keys and values at every layer.	Model weights untouched.
7	Prompt tuning	Learns soft prompt embeddings added at the input only.	Simplest; works better as models get larger.

> **Which one to pick (added).** Rule of thumb: LoRA/QLoRA is the default choice in practice. Others are worth knowing for exams and for special cases such as very small memory budgets.

4. Stage 2: fine-tuning at the data level

This split is based on **data preparation**. There are two types, and alignment (section 5) is the third stage.

Diagram 2: data-level approaches



Typical order: 1, then 2, then optional 3.

	Non-instructional (normal / domain adaptation)	Instructional (IFT / SFT)
Dataset	Plain text, such as company documents	Instruction + response format
Use case	Making the LLM a domain expert: medical, legal, financial documents	Chatbots, Q and A systems, tutors, coding assistants, customer support
Goal	Improve knowledge and understanding of a domain or language style	Teach the model to follow human instructions and act as an assistant
Output style	Text continuation; not necessarily able to follow instructions	Direct, helpful, structured answers, often with reasoning
Meaning	Learns the statistical distribution of text, not question-answering behavior	Learns Question to Answer or Prompt to Completion behavior
Common datasets	Your own documents	Alpaca, ShareGPT, Dolly, OpenOrca

Non-instructional fine-tuning: how the training rows look

The text is turned into many input to next-token pairs. Using the example from your class:

Input (given to the model)	Expected next text
Sunny	Savita
Sunny Savita	(dash)
Sunny Savita -	where
Sunny Savita - where	AI
Sunny Savita - where AI	meets
Sunny Savita - where AI meets	unstoppable
Sunny Savita - where AI meets unstoppable	passion

The same passage is used as a single plain-text training example, with no question and no answer.

Instruction fine-tuning: public conversational data sources

- Reddit (subreddits such as r/AskReddit, r/explainlikeimfive, r/askscience).
- Quora (human Q and A style).
- StackOverflow and StackExchange (technical question-answer patterns).
- GitHub Issues and Discussions (coding and troubleshooting threads).
- Open forums and community boards (Hugging Face, Kaggle, and so on).

Example IFT dataset row (from your notes)

```
Instruction: "Explain what mitochondria does in one sentence."
Response:    "Mitochondria generate energy for the cell through respiration."
```

> **Decision rule.** If the goal is to **add knowledge**, use normal fine-tuning. If the goal is a **chatbot or assistant**, use instruction fine-tuning. If you want both, do **domain fine-tuning first, then instruction fine-tuning**.

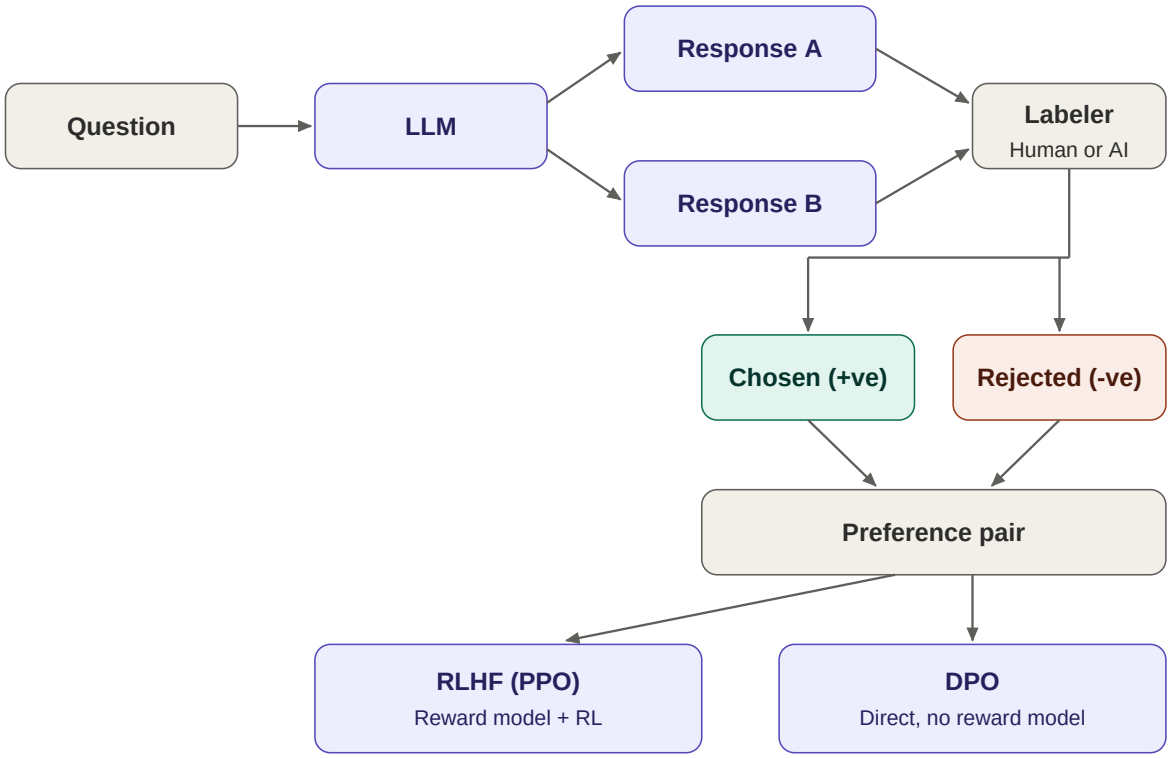
5. Stage 3: alignment with human feedback

Also called preference-based learning. It aligns the model's **responses** with what people prefer. Goal: make the model polite, safe, helpful and aligned with human values.

The feedback loop

Question a prompt > **LLM** writes 2+ responses > **Labeler** human or AI > **+ve / -ve** chosen vs rejected

Diagram 3: preference feedback flow



Positive (+ve) = chosen response; negative (-ve) = rejected response.

- **Data:** pairs of responses ranked by humans, or by AI models simulating human preference.
- **Positive (+ve)** = the chosen (better) response. **Negative (-ve)** = the rejected (worse) response.
- Together they form a **preference pair**: (prompt, chosen, rejected).

Algorithms

Method	Full name	How it works
RLHF	Reinforcement Learning from Human Feedback (PPO-based). Lineage in your notes: RL, then PPO (Proximal Policy Optimization), then RLHF, popularized by OpenAI.	(added) 1) Start from the SFT model. 2) Train a reward model on preference pairs. 3) Optimize the LLM with PPO to maximize reward, with a penalty that keeps it close to the SFT model.
DPO	Direct Preference Optimization. Lineage in your notes: supervised learning, then DPO.	(added) Trains directly on the preference pairs with a supervised-style loss that raises the likelihood of the chosen answer relative to the rejected one. No separate reward model and no RL loop, so it is simpler and cheaper.
RLAIF	Reinforcement Learning from AI Feedback.	An AI model does the ranking instead of humans.

Where preference data comes from (per your notes)

- **Real user logs:** collected with consent and anonymization; used to see which answer style users like more.
- **Human labeler feedback:** contractors worldwide rank better vs worse responses; helps train the reward model or preference pairs.

6. Family-wise breakdown of real models

Family	What happened at each stage	Open?
LLaMA (Meta)	Stage 1 pretraining on trillions of tokens. Stage 2 SFT made LLaMA-2-Chat. Stage 3 RLHF polished chat alignment. Meta released both base and chat variants, so you can choose raw or aligned.	Open weights
GPT (OpenAI)	GPT-3 (2020): pretrained base only. InstructGPT: SFT + RLHF. GPT-3.5 / GPT-4 (2022-23): heavily aligned with RLHF and safety filters.	Closed; no base weights
Mistral	Mistral-7B (2023): pretrained base. Mixtral-8x7B (MoE, 2023): pretrained mixture-of-experts. Instruct versions add SFT; some are also aligned with RLHF or DPO.	Open weights
DeepSeek (China)	Focus on efficient training and domain specialization (coding, reasoning). Large pretrained models. DeepSeek-Coder: pretrained + SFT on code data. DeepSeek-R1: trained with reinforcement signals (reasoning traces, self-improvement).	Open weights

Alignment examples (your notes)	Recipe
GPT-4	Pretrained GPT + SFT + RLHF
Gemini (Google)	SFT + RLHF + multimodal alignment
DeepSeek R1	Reinforcement-style fine-tuning with preference data

(check) Differences to verify. (1) The notes date InstructGPT to 2021; the paper was published in early 2022. (2) DeepSeek-R1 is best known for large-scale reinforcement learning with rule-based rewards (using an algorithm called GRPO) rather than mostly human preference pairs, so treat the "preference data" line as a simplification. (3) In your notes, item 1 under PEFT has no name; it is written here as LoRA / QLoRA based on its description.

7. Combo strategy used in industry

Step	Method	What you do
1	Normal fine-tuning	Adapt LLaMA to your domain (for example pharmaceutical documents)
2	Instruction fine-tuning	Train on user question-answer or task-instruction data
3 (optional)	RLHF / DPO	Refine using real user feedback and preference ranking

8. Worked example (added): a pharmaceutical assistant end to end

Goal: turn an open base model (LLaMA-style, 7B) into a chatbot that answers staff questions about company drug documents. GPU budget: one 24 GB card, so we use QLoRA.

Step 0: start from the base model

The base model was pretrained on web text. Ask it "What is the storage temperature for Product X?" and it may just continue the sentence or ramble, because it has never seen your documents and was never taught to answer questions.

Step 1: non-instructional fine-tuning (domain adaptation)

Data: plain text from product leaflets, trial reports and SOPs. No questions, no answers. One training row is just a chunk of text:

```
"Product X tablets must be stored below 25 C in the original blister pack,
 protected from moisture. Do not use after the expiry date printed on the carton..."
```

Learning signal: predict each next token. After this, the model *knows* the vocabulary and facts of your domain, but still just continues text.

Step 2: instruction fine-tuning

Data: a few thousand instruction and response pairs, written by experts or converted from documents.

```
Instruction: "What is the storage condition for Product X?"
Response:    "Store Product X below 25 C in its original blister pack, away from moisture."
```

Now the model answers questions directly instead of continuing text.

Step 3 (optional): alignment with preference data

For the same question the model produces two answers and a pharmacist marks one better:

```
Prompt:   "Can I give Product X to a child?"
Chosen  (+ve): "Product X is not approved for children under 12. Please consult a
                doctor or pharmacist before giving it."
Rejected (-ve): "Sure, just halve the dose."
```

Train with DPO (cheaper, no reward model) or RLHF with PPO. The model learns to prefer safe, careful, helpful answers.

How the efficient training is set up (QLoRA)

```
# Pseudocode using Hugging Face transformers + peft
model = load_model("llama-7b", load_in_4bit=True)      # lower-precision, memory-efficient loading
lora  = LoraConfig(r=16, lora_alpha=32, target_modules=["q_proj","v_proj"])
model = get_peft_model(model, lora)                    # only the small LoRA matrices train
train(model, dataset)                                  # step 1 data, then step 2 data
```

After step	Model behavior on "Storage for Product X?"
0 Base	Rambles or continues the sentence; may invent facts
1 Domain FT	Uses correct domain terms, but continues text instead of answering
2 Instruction FT	Gives a direct, correct answer in a helpful format
3 Alignment	Same answer plus safe, careful behavior on risky questions

9. Quick revision

- Pretraining is **self-supervised** next-token prediction and produces a **base model**.
- SFT has two lenses: **parameter level** (full vs partial) and **data level** (non-instructional vs instructional).
- Full fine-tuning updates all weights (heavy GPU need); partial = old school layer freezing or **PEFT**.
- PEFT list: LoRA/QLoRA, DoRA, Adapter layers, BitFit, IA3, Prefix tuning, Prompt tuning.
- Normal fine-tuning adds **knowledge**; instruction fine-tuning adds **assistant behavior**.

- Alignment uses **preference pairs** (+ve chosen, -ve rejected). RLHF = reward model + PPO; DPO = direct, no reward model; RLAIF = AI labeler.
- Industry order: domain fine-tune, then instruction fine-tune, then optional RLHF/DPO.

Self-test questions

- Why does a base model not answer questions well?
- What is the difference between a dataset row for normal fine-tuning and for IFT?
- Why does QLoRA use less memory than full fine-tuning?
- Name one advantage of DPO over RLHF.
- In what order would you apply domain FT, IFT and RLHF, and why?